Networking Temelleri - 2

---

1. Ağ Temellerine Giriş

1.1. Ağ Nedir?

Ağ (network), iki veya daha fazla cihazın veri alışverişi yapabilmek için birbirine bağlanmasıdır. Oyun perspektifinden bakıldığında ağ, oyuncuların birbirleriyle etkileşime girmesini sağlayan altyapıdır.

1.2. Ağ Türleri

Ağ Türü	Açıklama	Oyunlardaki Kullanımı
LAN (Local Area Network)	Aynı fiziksel mekandaki cihazların bağlantısı (ev, ofis, internet kafe)	Yerel multiplayer, LAN partileri (CS 1.6, Age of Empires)
WAN (Wide Area Network)	Geniş coğrafi alana yayılmış ağlar (şehirler, ülkeler arası)	İnternet üzerinden online oyunlar
MAN (Metropolitan Area Network)	Bir şehir ölçeğinde ağ	Şehir içi turnuvalar, e-spor etkinlikleri
PAN (Personal Area Network)	Kişisel cihazlar arası ağ (Bluetooth, NFC)	Mobil oyunlarda yakın cihaz bağlantısı

1.3. Ağ Topolojileri (Oyun Sunucuları İçin)

- **Star (Yıldız):** Tüm cihazlar merkezi bir sunucuya bağlanır. Çoğu online oyun bu topolojiyi kullanır.
- **Mesh (Örgü):** Her cihaz birbirine bağlıdır. Peer-to-peer (P2P) oyunlarda görülür.
- **Client-Server:** En yaygın model. Sunucu otorite, istemciler bağlıdır.

---

2. IP, Port ve Socket Kavramları

2.1. IP Adresi (Internet Protocol Address)

IP adresi, bir ağa bağlı her cihaza atanan benzersiz sayısal etikettir.

IPv4 Adres Yapısı:

```
192.168.1.100
|----|----|----|----|
  192   168   1   100
 (8 bit)(8 bit)(8 bit)(8 bit) = 32 bit toplam
```

IPv4 Adres Sınıfları:

Sınıf	Aralık	Kullanım
A	1.0.0.0 - 126.255.255.255	Büyük ağlar, az sayıda ağ, çok sayıda cihaz

Sınıf	Aralık	Kullanım
B	128.0.0.0 - 191.255.255.255	Orta ölçekli ağlar
C	192.0.0.0 - 223.255.255.255	Küçük ağlar (ev, ofis)
D	224.0.0.0 - 239.255.255.255	Multicast (çoklu yayın)
E	240.0.0.0 - 255.255.255.255	Deneysel/ayrılmış

Özel (Private) IP Adresleri:

- `10.0.0.0 – 10.255.255.255`
- `172.16.0.0 – 172.31.255.255`
- `192.168.0.0 – 192.168.255.255`

Bu adresler yerel ağlarda kullanılır ve internete doğrudan çıkamaz. Router (yönlendirici) üzerinden NAT (Network Address Translation) ile internete bağlanır.

Public vs Private IP:

Özellik	Public IP	Private IP
Tanım	İnternete doğrudan erişebilen benzersiz adres	Yerel ağ içinde kullanılan adres
Atama	ISP (Internet Servis Sağlayıcı) tarafından	Router tarafından (DHCP)
Örnek	`213.74.23.100`	`192.168.1.5`
Oyunlarda	Sunucu IP'si	Oyuncunun yerel cihaz IP'si

IPv6 Adres Yapısı:

```
2001:0db8:85a3:0000:0000:8a2e:0370:7334
|----|----|----|----|----|----|----|----|
 16   16   16   16   16   16   16   16  bit = 128 bit
```

IPv6'nın Avantajları:

- Çok daha fazla adres uzayı (2^{128})
- NAT'a gerek yok
- Daha basit başlık yapısı
- Otomatik yapılandırma desteği

2.2. Port Numarası (Port Number)

Port, bir cihazdaki belirli bir uygulamaya veya hizmete yönlendirme yapılmasını sağlar.

Port Aralıkları:

Aralık	Adı	Açıklama	Örnekler
0-1023	Well-Known Ports	Sistem tarafından ayrılmış	80 (HTTP), 443 (HTTPS), 21 (FTP), 22 (SSH)
1024-49151	Registered Ports	Kayıtlı uygulamalar	27015 (Valve), 25565 (Minecraft), 7777 (Unreal)
49152-65535	Dynamic/Private Ports	Geçici kullanım	İstemci uygulamalarının rastgele atadığı portlar

Oyunlarda Yaygın Portlar:

Oyun	Port	Protokol
Counter-Strike: GO	27015	UDP
Minecraft	25565	TCP
League of Legends	5000-5500	UDP
World of Warcraft	3724	TCP
Fortnite	5795-5847	UDP
Call of Duty	3074	UDP/TCP

2.3. Soket (Socket)

Soket, IP adresi ve port numarasının birleşimidir.

Soket Türleri:

Tür	Açıklama	Kullanım
Stream Socket (TCP)	Bağlantı tabanlı, güvenilir	Chat, dosya transferi, web
Datagram Socket (UDP)	Bağlantısız, hızlı	Oyun verileri, video streaming
Raw Socket	Düşük seviye, doğrudan IP paketleri	Ağ analizi, özel protokoller

Soket Yaşam Döngüsü (TCP):

```
1. socket()    → Soket oluştur
2. bind()      → IP ve port ata (sunucu)
3. listen()    → Bağlantıları dinle (sunucu)
4. accept()    → Bağlantıyı kabul et (sunucu)
5. connect()   → Sunucuya bağlan (istemci)
6. send()/recv() → Veri gönder/al
7. close()     → Bağlantıyı kapat
```

Soket Yaşam Döngüsü (UDP):

```
1. socket()    → Soket oluştur
2. bind()      → IP ve port ata (sunucu)
3. sendto()/recvfrom() → Veri gönder/al
4. close()     → Soketi kapat
```

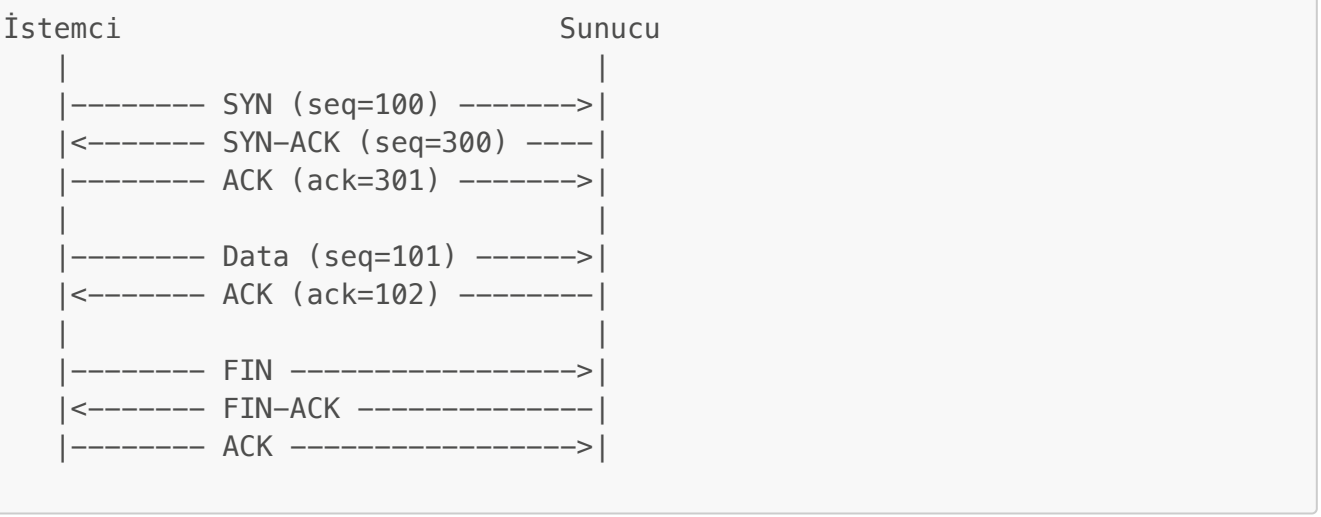
---

3. TCP vs UDP Farkları

3.1. TCP (Transmission Control Protocol)

TCP, güvenilir, sıralı ve bağlantı tabanlı bir protokoldür.

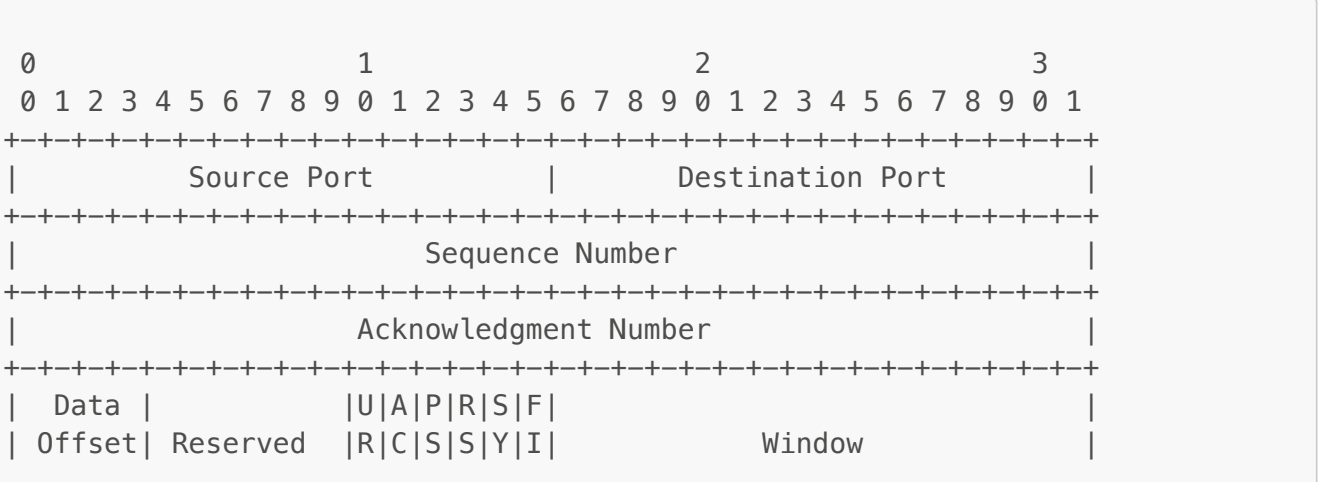
TCP'nin Çalışma Prensibi:



TCP'nin Özellikleri:

- **Bağlantı Tabanlı:** Veri göndermeden önce bağlantı kurulur.
- **Güvenilir:** Her paket için onay (ACK) beklenir.
- **Sıralı:** Paketler numaralandırılır, sırayla iletilir.
- **Akış Kontrolü:** Alıcının kapasitesine göre hız ayarlanır.
- **Tıkanıklık Kontrolü:** Ağ yoğunluğuna göre hız düşürülür.

TCP Başlık Yapısı (20 byte):



```
|       |           |G|K|H|T|N|N|                                 |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|           Checksum            |         Urgent Pointer        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

3.2. UDP (User Datagram Protocol)

UDP, hızlı, güvenilmez ve bağlantısız bir protokoldür.

UDP'nin Çalışma Prensibi:

```
İstemci                          Sunucu
   |                               |
   |-------- Data (paket 1) ------>|
   |-------- Data (paket 2) ------>|
   |-------- Data (paket 3) ------>|
   |                               |
   (Paket 2 kayboldu, kimse fark etmez)
```

UDP'nin Özellikleri:

- **Bağlantısız:** Önceden bağlantı kurulmaz.
- **Güvenilmez:** Paket kaybı kontrolü yok.
- **Sırasız:** Paketler farklı sırada gelebilir.
- **Hızlı:** Minimal overhead (8 byte başlık).
- **Hafif:** Düşük işlemci yükü.

UDP Başlık Yapısı (8 byte):

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|          Source Port          |       Destination Port        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|            Length             |           Checksum            |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

3.3. Detaylı Karşılaştırma Tablosu

Özellik	TCP	UDP
Bağlantı	Bağlantı tabanlı (3-way handshake)	Bağlantısız
Güvenilirlik	Yüksek (ACK, retransmission)	Düşük (best-effort)
Sıralama	Garantili	Garanti yok
Hız	Yavaş (overhead yüksek)	Hızlı (overhead düşük)

Özellik	TCP	UDP
Başlık Boyutu	20-60 byte	8 byte
Akış Kontrolü	Var	Yok
Tıkanıklık Kontrolü	Var	Yok
Kullanım Alanı	Web, e-posta, dosya transferi	Oyun, video, DNS, VoIP
Oyunlarda	Login, envanter, chat	Konum, ateş, hareket
Paket Kaybı	Telafi edilir	Tolere edilir
Gecikme	Yüksek	Düşük

3.4. TCP ve UDP'nin Avantaj/Dezavantajları

TCP Avantajları:

- Veri bütünlüğü garantisi
- Sıralı iletim
- Hata düzeltme
- Akış kontrolü

TCP Dezavantajları:

- Yüksek gecikme
- Yüksek bant genişliği kullanımı
- Bağlantı kurulumu zaman alır

UDP Avantajları:

- Düşük gecikme
- Yüksek hız
- Düşük bant genişliği kullanımı
- Bağlantı kurulumu gerekmez

UDP Dezavantajları:

- Paket kaybı
- Sırasız iletim
- Hata düzeltme yok
- Güvenilirlik yok

---

4. UDP vs TCP Kullanım Senaryoları

4.1. UDP'nin Kullanıldığı Senaryolar

Senaryo 1: Oyuncu Hareket Verileri

```
Durum: Bir FPS oyuncusu saniyede 60 kez konum güncelliyor.
Problem: Bir paket kaybolursa ne olur?
Çözüm: Bir sonraki paket zaten güncel konumu içerir.
Sonuç: UDP idealdir.
```

Senaryo 2: Ateş Etme ve Mermi Verileri

```
Durum: Oyuncu ateş ediyor, mermi verisi gönderiliyor.
Problem: Mermi verisi 100ms gecikirse ne olur?
Çözüm: Oyuncu "ben ateş ettim" hissini kaybeder.
Sonuç: UDP ile hızlı iletim şarttır.
```

Senaryo 3: Sesli İletişim (VoIP)

```
Durum: Oyuncular sesli sohbet ediyor.
Problem: Ses paketi kaybolursa ne olur?
Çözüm: Kısa bir çatırtı olur, konuşma devam eder.
Sonuç: UDP idealdir. TCP ile gecikmeli ses kabul edilemez.
```

4.2. TCP'nin Kullanıldığı Senaryolar

Senaryo 1: Oyun Girişi (Login)

```
Durum: Oyuncu kullanıcı adı ve şifre giriyor.
Problem: Şifre paketi kaybolursa ne olur?
Çözüm: Şifre eksik gider, giriş başarısız olur.
Sonuç: TCP ile güvenilir iletim şarttır.
```

Senaryo 2: Envanter ve Eşya Değişimi

```
Durum: Oyuncu eşya satın alıyor veya takas yapıyor.
Problem: İşlem paketi kaybolursa ne olur?
Çözüm: Eşya kaybolur veya çiftlenir (duplication bug).
Sonuç: TCP ile atomik işlem garantisi şarttır.
```

Senaryo 3: Oyun Güncellemesi İndirme

```
Durum: Oyuncu 5 GB'lık yama indiriyor.
Problem: Bir byte bozulursa ne olur?
Çözüm: Oyun çöker veya hatalı çalışır.
Sonuç: TCP (HTTP/HTTPS) ile bütünlük garantisi şarttır.
```

4.3. Hibrit Kullanım (Modern Oyunlar)

Veri Türü	Protokol	Neden?
Oyuncu konumu	UDP	Hızlı, kayıp tolere edilebilir
Ateş etme	UDP	Anlık, kayıp tolere edilebilir
Hasar	UDP	Hızlı, kayıp tolere edilebilir
Login	TCP	Güvenilir, kayıp kabul edilemez
Envanter	TCP	Güvenilir, kayıp kabul edilemez
Chat	TCP	Güvenilir, sıralı
Arkadaş listesi	TCP	Güvenilir
Maç sonu istatistik	TCP	Güvenilir

---

5. Çıkarım

Özet:

- IP adresi cihazı, port uygulamayı, soket ise ikisinin birleşimini tanımlar.
- TCP güvenilir ama yavaş, UDP hızlı ama güvenilmezdir.
- Oyunlarda genellikle hibrit yaklaşım kullanılır.
- Anlık veriler UDP, kritik veriler TCP ile gönderilir.

Soru-Cevap:

1. Neden oyunlarda TCP yerine UDP tercih edilir?
2. Bir oyunda hangi veriler TCP ile gönderilmelidir?
3. Port numarası olmadan iletişim mümkün müdür?

---

6. Basit TCP Chat Uygulaması

6.1. Uygulama Mimarisi

```
[İstemci 1] ---+
               |
[İstemci 2] ---+---> [TCP Sunucu] ---> Broadcast
               |
[İstemci 3] ---+
```

Mimari Bileşenler:

1. **TCP Sunucu:** Bağlantıları kabul eder, mesajları alır ve tüm istemcilere iletir.
2. **TCP İstemci:** Sunucuya bağlanır, mesaj gönderir ve alır.

6.2. TCP Sunucu Kodu

```
import socket
import threading

# ============================================================
# SUNUCU AYARLARI
# ============================================================
HOST = '127.0.0.1'  # Localhost - sadece kendi bilgisayarınızdan erişim
# HOST = '0.0.0.0'  # Tüm ağ arayüzlerinden erişim (gerçek sunucu için)
PORT = 12345        # Dinlenecek port (1024-49151 arası önerilir)
BUFFER_SIZE = 1024  # Bir seferde alınacak maksimum byte

# ============================================================
# GLOBAL DEĞİŞKENLER
# ============================================================
clients = []        # Bağlı istemcilerin socket listesi
nicknames = []      # İstemci takma adları (sıralı, clients ile eşleşir)

# ============================================================
# YARDIMCI FONKSİYONLAR
# ============================================================

def broadcast(message, sender_client=None):
    """
    Gönderici hariç tüm bağlı istemcilere mesajı iletir.

    Args:
        message (bytes): Gönderilecek mesaj (byte formatında)
        sender_client (socket): Mesajı gönderen istemci (hariç tutulur)
    """
    for client in clients:
        if client != sender_client:
            try:
                client.send(message)
            except:
                # Hata durumunda istemciyi listeden kaldır
                remove_client(client)

def remove_client(client):
    """
    Bağlantısı kopan istemciyi listeden temizler ve diğerlerine bildirir.

    Args:
        client (socket): Kaldırılacak istemci socket'i
    """
    if client in clients:
        index = clients.index(client)
        clients.remove(client)
        client.close()

        # Takma adı al ve listeden çıkar
        nickname = nicknames.pop(index) if index < len(nicknames) else
"Bilinmeyen"
```

```
        # Diğer istemcilere bildir
        broadcast(f"[SİSTEM] {nickname} sohbetten ayrıldı.".encode('utf-
8'))

def handle_client(client):
    """
    Her istemci için ayrı bir thread'de çalışır.
    İstemciden gelen mesajları sürekli dinler ve broadcast eder.

    Args:
        client (socket): İşlenecek istemci socket'i
    """
    while True:
        try:
            # İstemciden mesajı al
            message = client.recv(BUFFER_SIZE)

            if not message:
                # Bağlantı koptu
                remove_client(client)
                break
            else:
                # Alınan mesajı diğer herkese ilet
                broadcast(message, client)

        except ConnectionResetError:
            # İstemci aniden bağlantıyı kesti
            remove_client(client)
            break
        except Exception as e:
            print(f"[HATA] handle_client: {e}")
            remove_client(client)
            break

# ============================================================
# ANA SUNUCU FONKSİYONU
# ============================================================

def start_server():
    """
    Sunucuyu başlatır, bağlantıları kabul eder ve yönetir.
    """
    # TCP soketi oluştur
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    # SO_REUSEADDR: Port'un hemen yeniden kullanılmasını sağlar
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    # Soketi IP ve port'a bağla
    server.bind((HOST, PORT))

    # Bağlantıları dinlemeye başla (max 5 bekleyen bağlantı)
    server.listen(5)
```

```
    print(f"[BAŞLATILDI] Sunucu {HOST}:{PORT} adresinde çalışıyor...")
    print("[BİLGİ] Bağlantılar bekleniyor...\n")

    while True:
        # Yeni bağlantıyı kabul et (blocking)
        client, address = server.accept()
        print(f"[YENİ BAĞLANTI] {address}")

        # İstemciden kullanıcı adını al
        client.send("KULLANICI_ADI".encode('utf-8'))
        nickname = client.recv(BUFFER_SIZE).decode('utf-8')

        # Listelere ekle
        nicknames.append(nickname)
        clients.append(client)

        print(f"[KATILIM] {nickname} ({address}) sohbete katıldı.")
        print(f"[DURUM] Aktif istemci sayısı: {len(clients)}\n")

        # Katılım mesajını herkese duyur
        broadcast(f"[SİSTEM] {nickname} sohbete katıldı!".encode('utf-8'),
client)

        # Yeni istemciye hoş geldin mesajı gönder
        client.send(f"[SİSTEM] Sunucuya bağlandınız! Hoş geldiniz,
{nickname}.".encode('utf-8'))

        # Bu istemci için yeni bir thread başlat
        thread = threading.Thread(target=handle_client, args=(client,))
        thread.daemon = True  # Sunucu kapanınca thread de kapansın
        thread.start()

# ============================================================
# PROGRAM BAŞLANGICI
# ============================================================

if __name__ == "__main__":
    try:
        start_server()
    except KeyboardInterrupt:
        print("\n[KAPATILIYOR] Sunucu kapatılıyor...")
```

6.3. TCP İstemci Kodu

```
import socket
import threading
import sys

# ============================================================
# İSTEMCİ AYARLARI
# ============================================================
```

```python
HOST = '127.0.0.1'  # Sunucu IP adresi
PORT = 12345        # Sunucu port numarası
BUFFER_SIZE = 1024  # Bir seferde alınacak maksimum byte

# ============================================================
# MESAJ ALMA FONKSİYONU (THREAD)
# ============================================================

def receive_messages(client):
    """
    Sunucudan gelen mesajları sürekli dinler ve ekrana yazdırır.
    Ayrı bir thread'de çalışır.

    Args:
        client (socket): Sunucu bağlantı socket'i
    """
    while True:
        try:
            # Sunucudan mesaj al
            message = client.recv(BUFFER_SIZE).decode('utf-8')

            if message == "KULLANICI_ADI":
                # Sunucu kullanıcı adı istiyor
                nickname = input("Kullanıcı adınızı girin: ")
                client.send(nickname.encode('utf-8'))
            else:
                # Gelen mesajı ekrana yaz
                print(message)

        except ConnectionResetError:
            print("\n[SİSTEM] Sunucu bağlantıyı kesti!")
            client.close()
            sys.exit(1)
        except Exception as e:
            print(f"\n[HATA] Bağlantı hatası: {e}")
            client.close()
            sys.exit(1)

# ============================================================
# ANA İSTEMCİ FONKSİYONU
# ============================================================

def start_client():
    """
    İstemciyi başlatır, sunucuya bağlanır ve
    mesaj gönderme/alma işlemlerini yönetir.
    """
    # TCP soketi oluştur
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    # Sunucuya bağlan
    try:
        client.connect((HOST, PORT))
        print(f"[BAĞLANDI] Sunucuya bağlanıldı: {HOST}:{PORT}")
```

```
        print("[BİLGİ] Mesaj yazıp Enter'a basın. Çıkmak için '/quit'
yazın.\n")
    except ConnectionRefusedError:
        print(f"[HATA] Sunucuya bağlanılamadı! ({HOST}:{PORT})")
        print("[BİLGİ] Sunucunun çalıştığından emin olun.")
        return
    except Exception as e:
        print(f"[HATA] Bağlantı hatası: {e}")
        return

    # Mesaj alma işlemini ayrı thread'de başlat
    receive_thread = threading.Thread(target=receive_messages, args=
(client,))
    receive_thread.daemon = True
    receive_thread.start()

    # Ana thread'de mesaj gönderme işlemi
    while True:
        try:
            message = input()

            # Çıkış komutu
            if message.lower() == '/quit':
                print("[ÇIKILIYOR] Bağlantı kapatılıyor...")
                client.close()
                break

            # Boş mesaj gönderme
            if not message.strip():
                continue

            # Mesajı sunucuya gönder
            client.send(message.encode('utf-8'))

        except KeyboardInterrupt:
            print("\n[ÇIKILIYOR] Kullanıcı tarafından durduruldu.")
            client.close()
            break
        except Exception as e:
            print(f"[HATA] Mesaj gönderilemedi: {e}")
            client.close()
            break

# ============================================================
# PROGRAM BAŞLANGICI
# ============================================================

if __name__ == "__main__":
    start_client()
```

6.4. Uygulamayı Çalıştırma Adımları

1. **Terminal 1:** `python server.py` çalıştırın.

2. **Terminal 2:** `python client.py` çalıştırın. Kullanıcı adı girin.
3. **Terminal 3:** `python client.py` çalıştırın. Farklı kullanıcı adı girin.
4. Herhangi bir terminalden mesaj yazın, diğerlerinde göründüğünü gözlemleyin.
5. `/quit` yazarak çıkın, diğer istemcilere ayrılma mesajının gittiğini görün.

6.5. Kodun Çalışma Mantığı (Adım Adım)

Adım	Sunucu	İstemci
1	`socket()` ile TCP soketi oluşturur	-
2	`bind()` ile IP:Port'a bağlanır	-
3	`listen()` ile bağlantı bekler	-
4	-	`socket()` ile TCP soketi oluşturur
5	-	`connect()` ile sunucuya bağlanır
6	`accept()` ile bağlantıyı kabul eder	-
7	`KULLANICI_ADI` mesajı gönderir	Mesajı alır, kullanıcı adı ister
8	Kullanıcı adını alır, listeye ekler	Kullanıcı adını gönderir
9	Hoş geldin mesajı gönderir	Mesajı alır, ekrana yazdırır
10	Her istemci için thread başlatır	Mesaj alma thread'i başlatır
11	Gelen mesajları broadcast eder	Mesaj gönderir/alır

---

7. TCP Chat Uygulamasından Çıkarımlar

7.1. Oyun Geliştirme Perspektifi

Prensip	Chat Uygulaması	Oyun Karşılığı
Server Authority	Sunucu tüm mesajları dağıtır	Sunucu tüm hareketleri doğrular
Broadcasting	Mesajlar tüm istemcilere gider	Konumlar tüm oyunculara gider
Threading	Her istemci için ayrı thread	Her oyuncu için ayrı işlem
Connection Handling	Bağlantı kopunca temizlik	Oyuncu çıkınca state güncelleme
Message Protocol	Düz metin + özel komutlar	JSON/Binary + paket türleri

7.2. Sınırlamalar ve Gerçek Dünya Farkları

Chat Uygulaması	Gerçek Oyun
Düz metin mesajlar	Binary/JSON paketler
Az sayıda istemci (2-10)	Yüzlerce/binlerce istemci
Basit broadcast	Seçici broadcast (görüş alanı)

Chat Uygulaması	Gerçek Oyun
Thread-per-client	Event-driven / Async I/O
TCP (güvenilir)	UDP (hızlı) + TCP (kritik)
Basit hata yönetimi	Kapsamlı hata ve lag yönetimi

---

8. UDP vs TCP Karşılaştırmalı Uygulama

8.1. UDP Chat Uygulaması (Basit Gönderim)

UDP'nin bağlantısız yapısını göstermek için basit bir UDP chat uygulaması:

```
# ============================================================
# UDP CHAT SUNUCU (udp_server.py)
# ============================================================
import socket

HOST = '127.0.0.1'
PORT = 12345
BUFFER_SIZE = 1024

# UDP soketi oluştur
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind((HOST, PORT))
print(f"[UDP SUNUCU] {HOST}:{PORT} adresinde çalışıyor...")

# Bağlı istemcilerin adreslerini tut
clients = set()

while True:
    try:
        # Veriyi ve gönderen adresini al
        data, client_addr = server.recvfrom(BUFFER_SIZE)

        # Yeni istemci ise listeye ekle
        if client_addr not in clients:
            clients.add(client_addr)
            print(f"[YENİ] {client_addr} bağlandı. Toplam: {len(clients)}")

        # Mesajı gönderen hariç tüm istemcilere ilet
        message = data.decode('utf-8')
        print(f"[{client_addr}] {message}")

        for c in clients:
            if c != client_addr:
                server.sendto(data, c)

    except KeyboardInterrupt:
        print("\n[KAPATILIYOR] Sunucu kapatılıyor...")
        break
    except Exception as e:
```

```
        print(f"[HATA] {e}")

server.close()
```

```
# ============================================================
# UDP CHAT İSTEMCİ (udp_client.py)
# ============================================================
import socket
import threading

HOST = '127.0.0.1'
PORT = 12345
BUFFER_SIZE = 1024

def receive_messages(client_socket):
    """Sunucudan gelen UDP paketlerini dinler."""
    while True:
        try:
            data, _ = client_socket.recvfrom(BUFFER_SIZE)
            print(f"\n{data.decode('utf-8')}")
            print("> ", end="", flush=True)
        except:
            break

# UDP soketi oluştur
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Alıcı thread'i başlat
receive_thread = threading.Thread(target=receive_messages, args=(client,))
receive_thread.daemon = True
receive_thread.start()

# Kullanıcı adı al
nickname = input("Kullanıcı adınız: ")
client.sendto(f"[KATILIM] {nickname}".encode('utf-8'), (HOST, PORT))

print(f"[UDP İSTEMCİ] Sunucuya bağlanıldı: {HOST}:{PORT}")
print("Mesaj yazın ('quit' ile çıkın):\n")

while True:
    message = input("> ")
    if message.lower() == 'quit':
        client.sendto(f"[AYRILMA] {nickname}".encode('utf-8'), (HOST,
PORT))
        break

    full_message = f"{nickname}: {message}"
    client.sendto(full_message.encode('utf-8'), (HOST, PORT))

client.close()
```

8.2. TCP vs UDP Chat Karşılaştırması

Özellik	TCP Chat	UDP Chat
Bağlantı	3-way handshake gerekli	Bağlantı yok
İstemci Takibi	Socket listesi	IP:Port seti
Mesaj Kaybı	Asla kaybolmaz	Kaybolabilir
Mesaj Sırası	Garantili	Garanti yok
Hata Tespiti	Bağlantı kopunca anında	Kayıp paket fark edilmez
Kod Karmaşıklığı	Orta	Basit
Performans	Daha yavaş	Daha hızlı

8.3. UDP'de Gözlemlenebilecek Sorunlar

1. **Paket Kaybı:** Mesajlar bazen karşı tarafa ulaşmaz.
2. **Sıra Bozukluğu:** Mesajlar farklı sırada gelebilir.
3. **Bağlantı Kopma Tespiti:** İstemci kapansa bile sunucu fark etmez.
4. **Hayalet İstemciler:** Ayrılan istemciler listede kalır.

**9. UYGULAMA ÖDEVİ **

Ödev 1: TCP Chat'e Zaman Damgası Ekleme

Her mesaja gönderim zamanını ekleyin:

```
[14:30:25] Ali: Merhaba arkadaşlar!
[14:30:28] Ayşe: Merhaba Ali!
```

Ödev 2: TCP Chat'e Özel Mesaj Ekleme

`/msg <kullanıcı_adı> <mesaj>` komutu ile özel mesaj gönderme:

```
/msg Ayşe Merhaba, nasılsın?
```

Ödev 3: UDP Chat'e "Güvenilirlik" Katmanı Ekleme

Her mesaja bir sıra numarası ekleyin ve kayıp mesajları tespit edin.

Ödev 4: Hibrit Chat Uygulaması

- Login işlemi TCP ile
- Mesajlaşma UDP ile
- Sistem bildirimleri TCP ile

---

10. Özet

10.1. Öğrenilen Kavramlar

Kavram	Açıklama
IP Adresi	Cihazın ağdaki benzersiz adresi
Port	Cihazdaki uygulamaya yönlendirme
Socket	IP + Port = İletişim kapısı
TCP	Güvenilir, sıralı, bağlantı tabanlı
UDP	Hızlı, güvenilmez, bağlantısız
Client-Server	Sunucu otorite, istemciler bağlı
Broadcast	Bir mesajın tüm istemcilere gönderilmesi
Threading	Eşzamanlı işlem yönetimi

10.2. Oyun Geliştirmede Kullanım

- **Anlık veriler (konum, ateş):** UDP
- **Kritik veriler (login, envanter):** TCP
- **Hibrit yaklaşım:** Modern oyunların standardı
- **Server authority:** Hile önleme için şart

10.3. Sonraki Dersler İçin Temel

Bu derste öğrendiklerimiz, ilerleyen derslerde göreceğimiz şu konulara temel oluşturur:

- Serialization (JSON, Binary)
- State Sync (Full, Delta)
- Game Loop ve Tick Rate
- Lag Compensation
- Client-Side Prediction

Kaynak	İçeriği
"Multiplayer Game Programming" - Glazer & Madhav	Kapsamlı oyun ağ programlama kitabı

Bir sonraki derste, veri serileştirme (serialization) ve mesaj formatı tasarımı konuları ele alınacak.